Question 1 : What is the difference between AI, ML, DL, and Data Science? Provide a brief explanation of each.

1. Artificial Intelligence (AI): The Grand Vision

AI is the quest to build machines that can perform tasks that typically require human intelligence. It is the "umbrella" under which everything else sits.

- **Scope:** Global. It includes not just learning from data, but also **Symbolic AI** (logic-based), **Robotics**, and **Natural Language Generation**.
- **Techniques:** * **Rule-based Systems:** Expert systems that follow "If-Then" logic (e.g., a medical diagnosis tool from the 1980s).
    - **Search Algorithms:** Pathfinding in maps (like A* algorithm).
- **Applications:** Smart assistants (Alexa), NPC behavior in video games, and automated manufacturing.
- **Detailed Insight:** AI doesn't always need data to be "smart." A calculator is a form of AI because it performs a human cognitive task, even though it doesn't "learn" or use Machine Learning.

2. Machine Learning (ML): The Engine of Prediction

Machine Learning is the realization of AI through the use of algorithms that parse data, learn from it, and then make a determination or prediction about something in the world.

- **Scope:** Focuses on the development of computer programs that can access data and use it to learn for themselves.
- **Techniques:**
    - **Supervised Learning:** Training on labeled data (e.g., $y = f(x)$).
    - **Unsupervised Learning:** Finding hidden patterns in unlabeled data (Clustering).
    - **Reinforcement Learning:** Learning through trial and error using rewards.
- **Applications:** Predictive maintenance in factories, dynamic pricing for Uber/Lyft, and personalized Netflix recommendations.
- **Detailed Insight:** In ML, **Feature Engineering** is crucial. Humans must often tell the machine which features (like "number of rooms" or "square footage") are important for the model to succeed.

---

3. Deep Learning (DL): The Power of Neural Networks

Deep Learning is a specialized sub-field of ML that uses multi-layered artificial neural networks to deliver state-of-the-art accuracy in tasks like object detection and speech recognition.

- **Scope:** Mimicking the human brain's neocortex. It requires massive amounts of data and significant computational power (GPUs).
- **Techniques:**
    - **Convolutional Neural Networks (CNN):** Specialized for processing pixel data (Images).
    - **Generative Adversarial Networks (GANs):** Used to create realistic images/art.
    - **Transformers:** The architecture behind Large Language Models like GPT.
- **Applications:** Self-driving cars (interpreting video feeds), real-time language translation (Google Translate), and medical imaging (detecting tumors).
- **Detailed Insight:** Unlike ML, DL performs **Automatic Feature Extraction**. You don't tell the model to look for "eyes" or "ears" in a photo; the "Deep" layers figure out those features on their own.

---

4. Data Science (DS): The Decision Science

Data Science is an interdisciplinary field that uses scientific methods, processes, and systems to extract knowledge and insights from structured and unstructured data.

- **Scope:** Broad and business-centric. It encompasses the entire data lifecycle: Collection -> Cleaning -> Analysis -> Visualization -> Decision.
- **Techniques:**
    - **Statistics & Probability:** Understanding data distributions and significance.
    - **Data Wrangling:** Cleaning "messy" data into a usable format.
    - **Data Visualization:** Using tools like Tableau or Matplotlib to tell a story.
- **Applications:** Detecting fraud in banking, optimizing supply chains, and identifying customer churn in telecom.
- **Detailed Insight:** A Data Scientist might use an ML model to predict sales, but their primary job is to explain **why** the sales are changing and how the business should pivot.

Question 2: Explain overfitting and underfitting in ML. How can you detect and prevent them?

1. Underfitting (High Bias)

Underfitting occurs when a model is too simple to capture the underlying structure of the data. It performs poorly on both the training data and the test data.

- **The Concept:** Imagine trying to fit a straight line through a set of data points that follow a curve. The model "misses" the relevant relations.
- **Bias:** Underfitting is associated with **High Bias**. This means the model makes strong (and incorrect) assumptions about the data.
- **Detection:** Low accuracy on the training set and low accuracy on the validation set.

2. Overfitting (High Variance)

Overfitting occurs when a model learns the training data "too well," including the noise and random fluctuations (outliers). While it performs perfectly on training data, it fails to generalize to new data.

- **The Concept:** Imagine a student who memorizes every specific answer in a textbook but fails the exam because the questions are phrased slightly differently.
- **Variance:** Overfitting is associated with **High Variance**. The model is overly sensitive to small fluctuations in the training set.
- **Detection:** High accuracy on the training set but a significant drop in accuracy on the test/validation set.

---

3. The Bias-Variance Tradeoff

The goal of any ML model is to find the "Sweet Spot" between bias and variance.

-
- As you increase model complexity (adding more features or layers), **Bias decreases** but **Variance increases**.
- The **Total Error** is a combination of $(Bias^2 + Variance + Irreducible Noise)$.
- An optimal model minimizes this total error by balancing the two.

---

4. How to Detect Overfitting and Underfitting

A. Learning Curves

By plotting the training error vs. validation error against the number of training iterations:

- If both errors are high $\rightarrow$ **Underfitting**.
- If training error is low but validation error is high (a large gap) $\rightarrow$ **Overfitting**.

B. K-Fold Cross-Validation

Instead of a single train-test split, the data is divided into k subsets. The model is trained k times, each time using a different subset as the test set. If the performance varies wildly across folds, the model is likely **overfitting**.

5. How to Prevent Them

To Prevent Underfitting (Reduce Bias):

1. **Increase Model Complexity:** Switch from a linear model to a non-linear one (e.g., from Linear Regression to a Polynomial Regression or Neural Network).

2. **Feature Engineering:** Add more relevant features or combine existing ones to give the model more information.
3. **Reduce Regularization:** If you are using penalties, decrease them to allow the model more flexibility.

To Prevent Overfitting (Reduce Variance):

1. **Regularization (L1 & L2):** * **L1 (Lasso):** Adds a penalty proportional to the absolute value of coefficients, often leading to sparse models (feature selection).
    - **L2 (Ridge):** Adds a penalty proportional to the square of the magnitude of coefficients, preventing any single feature from having too much influence.
2. **Early Stopping:** In iterative training (like Neural Networks), stop the training process as soon as the validation error starts to rise, even if the training error is still falling.
3. **Data Augmentation:** Increase the size of the training set by creating "synthetic" data (e.g., flipping or rotating images) to help the model learn general patterns rather than specific details.
4. **Dropout:** (Specific to Deep Learning) Randomly "turning off" neurons during training to prevent them from co-depending on each other.

Question 3:How would you handle missing values in a dataset? Explain at least three methods with examples ?

Handling missing values is one of the most critical steps in the **Data Preprocessing** pipeline. If left unaddressed, missing data can lead to biased estimates, reduced statistical power, and even model failure.

THE THREE METHODS FOR HANDLING MISSING VALUES ARE FOLLOWING:

1. Deletion (Listwise and Pairwise)

The simplest approach is to remove the missing data points entirely. This is generally divided into two types:

- **Listwise Deletion:** Removing the entire row (observation) if even one value is missing.
- **Column Deletion:** Removing the entire feature (variable) if it has a very high percentage of missing values (e.g., >60%).

Example: In a medical dataset of 1,000 patients, if 5 patients did not record their "Weight," those 5 rows are deleted.

> **Pros:** Fast and easy to implement. **Cons:** Significant loss of information. If the data is not "Missing Completely at Random" (MCAR), this method introduces severe bias.

---

2. Statistical Imputation (Mean, Median, Mode)

Instead of deleting data, we replace the missing value with a statistically calculated value from the rest of the column.

- **Mean Imputation:** Use the average of the column. Best for normally distributed (Bell Curve) numerical data.
- **Median Imputation:** Use the middle value. Best for numerical data with **outliers**, as the median is more robust than the mean.
- **Mode Imputation:** Use the most frequent value. Best for **categorical** data (e.g., "City" or "Gender").

Example:

In a real estate dataset, the "Number of Bedrooms" is missing for some houses. Since you can't have 3.4 bedrooms (mean), you use the **Mode** (e.g., 3) to fill the gaps.

> **Pros:** Maintains the sample size and is computationally efficient.
>
> **Cons:** It reduces the variance of the data and ignores relationships between variables.

---

3. Advanced Predictive Modeling (MICE / KNN)

This treats the missing value as a "Target Variable" to be predicted using the other available features in the dataset.

- **K-Nearest Neighbors (KNN):** The algorithm finds the 'k' most similar rows based on other features and uses their values to fill the gap.
- **MICE (Multivariate Imputation by Chained Equations):** A sophisticated method that models each variable with missing values as a function of other variables in an iterative process.

Example:

If a person's "Income" is missing, a predictive model looks at their "Education Level," "Years of Experience," and "Job Title" to estimate a highly accurate income value based on similar profiles in the data.

> **Pros:** Much more accurate than simple averages; preserves the relationship between features. **Cons:** Computationally expensive for large datasets and can lead to "data leakage" if not handled carefully during the train-test split.

Question 4:What is an imbalanced dataset? Describe two techniques to handle it (theoretical + practical).

1. Defining the Imbalanced Dataset

An **imbalanced dataset** occurs when the classification categories (classes) are not represented equally. One class (the **Majority Class**) significantly outnumbers the other (the **Minority Class**).

- **The Problem:** Standard ML algorithms are designed to maximize overall **Accuracy**. In an imbalanced scenario, a model can achieve 99% accuracy by simply predicting the majority class every time, completely failing to identify the minority class (which is often the class of interest).
- **Real-World Examples:** * **Fraud Detection:** 99.9% of transactions are legitimate; 0.1% are fraudulent.
    - **Disease Diagnosis:** In a screening, most patients are healthy; very few have the rare disease.

2. Technique 1: Resampling (SMOTE & Random Sampling)

Resampling involves shrinking the majority class or expanding the minority class to create a balanced 50/50 distribution.

A. Random Under-sampling (Theoretical)

You randomly delete examples from the majority class until it matches the size of the minority class.

- **Pros:** Reduces the size of the training data, making it faster to train.
- **Cons:** You may throw away valuable information that could have helped the model learn.

B. SMOTE: Synthetic Minority Over-sampling Technique (Practical/Advanced)

Instead of simply duplicating the minority rows (which causes overfitting), SMOTE creates **synthetic** (fake but realistic) examples.

- **How it works:** It selects a point in the minority class, finds its nearest neighbors, and creates new data points along the lines connecting them.
- **Practical Use:** Widely used in Python via the `imblearn` library. It ensures the model learns the "area" of the minority class rather than just memorizing specific points.

---

3. Technique 2: Algorithmic Adjustment (Class Weights)

Instead of changing the data, this technique changes how the **algorithm** treats the data during the training phase.

A. Cost-Sensitive Learning (Theoretical)

The idea is to "punish" the model more heavily for misclassifying a minority instance than a majority instance.

- **Analogy:** If a smoke detector misses a fire (Minority), it's a disaster. If it goes off for burnt toast (Majority), it's just an annoyance. We want the model to be terrified of missing the fire.

B. Balanced Class Weights (Practical)

Most modern ML libraries (like Scikit-Learn or XGBoost) have a parameter called class_weight='balanced'.

- **How it works:** It automatically calculates weights inversely proportional to class frequencies.
- **Equation:**
  $$W_j = \frac{n}{k \cdot n_j}$$
  (Where n is total samples, k is number of classes, and n_j is samples in class j.)
- This forces the loss function to pay more attention to the minority class during optimization.

Question 5: Why is feature scaling important in ML? Compare Min-Max scaling and Standardization.

Feature scaling is a critical preprocessing step that ensures all input variables are on a similar scale. Without it, machine learning models can become biased toward features with larger numerical values, leading to poor performance and slow training.

A. Impact on Distance-Based Algorithms

Algorithms like **K-Nearest Neighbors (KNN)** and **Support Vector Machines (SVM)** calculate the distance (usually Euclidean distance) between data points to make predictions.

- **The Problem:** If one feature has a range of 0–1,000,000 (like Annual Income) and another has a range of 0–10 (like Years of Experience), the "Income" feature will dominate the distance calculation. The model will essentially ignore the "Experience" feature.

- **The Solution:** Scaling ensures all features contribute equally to the distance.

B. Impact on Gradient Descent

For algorithms like **Linear Regression**, **Logistic Regression**, and **Neural Networks**, we use Gradient Descent to find the minimum of the cost function.

- **The Problem:** If features have vastly different scales, the cost function's contours will be tall and skinny (elliptical). This causes the gradient to "oscillate" or zig-zag, making it take much longer to reach the center (the global minimum).
- **The Solution:** Scaling makes the contours more spherical, allowing Gradient Descent to "smoothly" and quickly converge to the minimum.

2. Min-Max Scaling vs. Standardization

A. Min-Max Scaling (Normalization)

This technique rescales the data into a fixed range, usually **[0, 1]**.

- **Formula:**
  $$X_{norm} = \frac{X - X_{min}}{X_{max} - X_{min}}$$
- **Best Used When:** You know the distribution of your data is NOT Gaussian (normal), or when the algorithm requires values between 0 and 1 (like Image Processing or Neural Networks using Sigmoid activation).
- **Weakness:** It is highly sensitive to **outliers**. If you have one massive outlier, all other values will be squashed into a tiny range (e.g., 0 to 0.01).

B. Standardization (Z-Score Normalization)

This technique centers the data so that the **Mean = 0** and the **Standard Deviation = 1**.

- **Formula:**
  $$X_{std} = \frac{X - \mu}{\sigma}$$
  (Where μ is the mean and σ is the standard deviation.)
- **Best Used When:** The data follows a Gaussian (Normal) distribution. Many algorithms (like SVM, Logistic Regression, and PCA) assume that the data is centered around zero.
- **Strength:** It is much more **robust to outliers**. Even with extreme values, the rest of the data remains spread out in a way that the model can still interpret.

COMPARISON

Feature	Min-Max Scaling	Standardization

Output Range	Usually [0, 1] or [-1, 1].	No fixed range (usually -3 to 3).
Mean/Std Dev	Varies.	Mean = 0, Std Dev = 1.
Outlier Impact	Highly sensitive (Outliers squash data).	Less sensitive (Handles outliers better).
Common Use Case	Image processing, Neural Networks.	PCA, SVM, Linear/Logistic Regression.

Question 6: Compare Label Encoding and One-Hot Encoding. When would you prefer one over the other?

1. Label Encoding (Integer Encoding)

Label Encoding assigns a unique integer to each category in a feature.

- **How it works:** If you have a "Size" column with values [Small, Medium, Large], Label Encoding converts them to [0, 1, 2].
- **Best Used For: Ordinal Data**. This is data where the categories have a natural, inherent order or rank.
- **The Advantage:** It is computationally efficient and keeps the dataset's dimensionality (the number of columns) low.
- **The Trap:** If applied to non-ordered data (like colors), the model might mistakenly assume that "Green (2)" is greater than "Red (0)," leading to biased mathematical weightings.

---

2. One-Hot Encoding (Dummy Variables)

One-Hot Encoding creates a new binary column (0 or 1) for every unique category in the original feature.

- **How it works:** If you have a "Color" column with [Red, Green, Blue], it creates three new columns: Is_Red, Is_Green, and Is_Blue. For a "Red" row, the value is 1 in the Is_Red column and 0 in others.
- **Best Used For: Nominal Data**. This is data where there is no relationship or rank between categories (e.g., Countries, Zip Codes, or Gender).
- **The Advantage:** It ensures the model does not assume a mathematical order where none exists.
- **The Trap: The Curse of Dimensionality**. If a feature has 500 unique categories (High Cardinality), One-Hot Encoding adds 500 new columns, which can make the model extremely slow and prone to overfitting.

---

3. Comparison Summary Table

Feature	Label Encoding	One-Hot Encoding
Data Type	Ordinal (Ranked)	Nominal (No Rank)
Dimensionality	Stays the same (1 column)	Increases (N columns)
Model Impact	May introduce false order	Prevents false order
Example	Education (High School < PhD)	Country (India, USA, Brazil)

---

4. Practical Considerations for the Assignment

A. The Dummy Variable Trap

When using One-Hot Encoding, you should often drop one of the new columns ($N-1$ encoding). For example, in a "Gender" column (Male/Female), if Is_Male is 0, the model automatically knows it is Female. Keeping both causes **Multicollinearity**, which can mess up models like Linear Regression.

B. When to prefer which?

- **Choose Label Encoding:** When the categories follow a sequence (e.g., Cold, Warm, Hot) or when using **Tree-based models** (Random Forest, XGBoost), which can actually handle Label Encoding quite well even for nominal data.

- **Choose One-Hot Encoding:** When the categories are independent (e.g., Apple, Orange, Banana) and you are using **Linear models** (Logistic Regression, SVM) or Neural Networks.

Question 7: Google Play Store Dataset
a). Analyze the relationship between app categories and ratings. Which categories have the highest/lowest average ratings, and what could be the possible reasons?

1. Analysis of App Categories vs. Ratings

In most versions of this dataset (like the popular Kaggle version), the average rating across the store hovers around **4.1 to 4.3**. However, when grouped by category, distinct patterns emerge.

A. Categories with Highest Average Ratings

- **Common Categories: Education**, **Events**, **Health & Fitness**, and **Books & Reference**.
- **Average Rating:** Often ranges between **4.4 and 4.7**.
- **Possible Reasons:**
    - **High Utility & Specificity:** Users downloading an "Education" or "Health" app usually have a specific goal. If the app helps them learn a language or track calories, they perceive high value.
    - **Targeted Audience:** These apps aren't "mass market" in the same way games are; they attract a more focused, often more forgiving or appreciative user base.
    - **Lower Complexity:** Many "Events" or "Reference" apps are content-heavy but technically simple, leading to fewer bugs and crashes—the primary drivers of 1-star reviews.

B. Categories with Lowest Average Ratings

- **Common Categories: Dating**, **Maps & Navigation**, and **Tools**.
- **Average Rating:** Often ranges between **3.8 and 4.0**.
- **Possible Reasons:**
    - **High Technical Friction:** "Maps" and "Tools" (like cleaners or battery savers) are prone to technical glitches, high battery consumption, or intrusive ads, which immediately result in poor ratings.
    - **Emotional Factor (Dating):** Ratings in the "Dating" category are often influenced by the user's personal success on the platform. If a user doesn't get matches, they are likely to leave a frustrated 1-star review, regardless of the app's technical quality.
    - **Monetization:** "Tools" are notorious for "Aggressive Monetization" (pop-up ads), which is the number one complaint in the Play Store comments.

THE RATING PARADOX

- **Sample Size Bias:** Small categories with only a few apps (like "Events") can have skewed averages because a single 5-star app moves the needle significantly.
- **The "Free vs. Paid" Influence:** Generally, paid apps have slightly higher ratings because users who pay are more committed to the app, and developers of paid apps usually provide better support and fewer ads.
- **App Size & Performance:** There is often a negative correlation between very large app sizes and ratings in regions with poor internet connectivity, affecting "Games" and "Family" categories more than others.

CODE UPLOADED ON THAT LINK

Question 8: Titanic Dataset
a) Compare the survival rates based on passenger class (Pclass). Which class had the highest survival rate, and why do you think that happened?
b) Analyze how age (Age) affected survival. Group passengers into children (Age < 18) and adults (Age ≥ 18). Did children have a better chance of survival?

1. Introduction: The Socio-Economic Context

The Titanic dataset is more than just numbers; it is a snapshot of 1912 social stratification. To analyze survival, we must look at the **intersectionality** of class, age, and gender. The "Women and Children First" protocol (The Birkenhead Drill) acted as a non-random filter that overrides pure statistical probability.

---

2. Part A: Impact of Passenger Class (Pclass) on Survival

The Statistical Reality

- **Pclass 1 (Elite):** Survival rate ~63%.
- **Pclass 2 (Middle):** Survival rate ~47%.
- **Pclass 3 (Lower):** Survival rate ~24%.

Reasons for the Disparity:

1. **Structural Architecture (The "Deck" Factor):** 1st-class cabins were located on the **Boat Deck** and **A-Deck**. 3rd-class passengers were housed in the lower "Steerage" decks. During the sinking, the physical distance and the maze of corridors significantly delayed 3rd-class access to lifeboats.
2. **Information Asymmetry:** Stewards were assigned specifically to 1st-class areas to wake passengers and lead them to boats. 3rd-class passengers were often left to find their own way, with some gates reportedly being locked early in the night.

3. **Lifeboat Capacity:** There were only enough lifeboats for about 53% of the people on board. Naturally, those closest to the launching points (the wealthy) filled them first.

3. Part B: The "Age" Variable (Children vs. Adults)

The Categorical Analysis

we define "Children" as anyone under 18. This group historically shows a survival rate of **~54%**, compared to roughly **36%** for adults.

Why Children had a Higher Probability:

- **The Moral Code:** The Captain's order of "Women and Children First" was strictly enforced by officers like Lightoller. In many instances, men were prohibited from entering boats even if there was empty space, provided children were still on deck.
- **The "Weight" Factor:** In the final stages of the sinking, children were easier to pass into crowded boats or lower into them via ropes compared to adult men.
- **Class Nuance:** It is important to note that a child in 1st class had a near 100% survival rate, whereas a child in 3rd class had a much lower chance—showing that **Class** often trumped **Age** in terms of survival priority.

CODE UPLOADED

Question 9: Flight Price Prediction Dataset

a) How do flight prices vary with the days left until departure? Identify any exponential price surges and recommend the best booking window.

b)Compare prices across airlines for the same route (e.g., Delhi-Mumbai). Which airlines are consistently cheaper/premium, and why?

9a) Flight Prices vs. Days Left Until Departure

The Analysis: The "J-Curve" Effect

In the Flight Price dataset, there is a strong negative correlation between "Days Left" and "Price." As the days left decrease, the price increases, but not linearly.

- **The Trend:** Prices remain relatively stable or "flat" when booking **30 to 60 days** in advance.

- **The Exponential Surge:** A massive price surge typically begins **15 days before departure**, with a second, even steeper "exponential" spike occurring within the **last 48 hours**.
- **Reasoning:** Airlines use **Yield Management Systems**. They know that "Leisure Travelers" book early and are price-sensitive, while "Business Travelers" book late and are price-insensitive (their company pays). The last-minute spike captures the high urgency of business needs.

Recommendation: The Best Booking Window

- **The "Sweet Spot":** For the best balance of availability and price, travelers should book between **21 and 42 days** before departure.
- **The Risk Zone:** Avoid booking within **7 days** of departure, as prices can be $2\times$ or $3\times$ higher than the baseline.

9b) Airline Price Comparison (Delhi-Mumbai)

The Analysis: Low-Cost Carriers (LCC) vs. Full-Service Carriers (FSC)

When analyzing a high-traffic route like Delhi-Mumbai, airlines generally fall into two statistical clusters.

Airline Category	Examples (Dataset Specific)	Price Trend	Reason for Pricing
Budget/ LCC	AirAsia, Indigo, SpiceJet	**Cheaper**	High aircraft utilization, point-to-point travel, and "unbundled" services (pay for meals/bags).
Premium/FSC	Air India, Vistara	**Premium**	"Bundled" services (free meals, more legroom, lounge access), and higher operational costs.

Which is consistently cheaper?

- **Cheaper: AirAsia** and **Indigo** usually offer the lowest base fares. They operate on a "Volume" model, keeping seats filled at lower prices.
- **Premium: Vistara** and **Air India** consistently show higher median prices. Vistara, in particular, positions itself as a premium brand with a "Business Class" and "Premium Economy" section that inflates the average route price.

CODE UPLOADED

Question 10: HR Analytics Dataset

a). What factors most strongly correlate with employee attrition? Use visualizations to show key drivers (e.g., satisfaction, overtime, salary).

b). Are employees with more projects more likely to leave?

10a) Factors Strongly Correlated with Attrition

The Analysis: Top 3 Drivers

1. **Overtime (The Burnout Factor):** This is often the strongest predictor. Employees who frequently work overtime have a significantly higher attrition rate. It indicates a poor work-life balance.
2. **Monthly Income (The Financial Factor):** There is an inverse correlation here; as salary increases, the probability of leaving decreases. However, there is a "mid-level trap" where mid-career employees leave for better competitive offers.
3. **Job Satisfaction & Environment:** Lower scores in "Job Satisfaction," "Relationship Satisfaction," and "Work-Life Balance" surveys correlate directly with the "Yes" category for Attrition.

10b) Project Load vs. Attrition

The "U-Shaped" Relationship

When analyzing the number of projects (`number_project`) or total hours worked, the relationship is usually not a straight line, but a **U-curve**:

- **Low Projects (2 or fewer):** These employees are **highly likely to leave**. They feel underutilized, bored, or may be "quiet quitting" before officially resigning.

- **Optimal Projects (3 to 4):** This is the "Sweet Spot." These employees show the lowest attrition rates as they are engaged but not overwhelmed.
- **High Projects (5 or more):** Attrition **spikes again**. These employees are likely experiencing burnout and high stress, leading them to seek roles with more manageable workloads.

Conclusion: Yes, employees with *too many* projects are more likely to leave, but so are those with *too few*.

CODE UPLOADED